Abstract xxxx

1 Introduction

xxxx

2 Background and Motivation

2.1 Data Stealing

2.2 Motivation

3 Empirical Study Setup

In this study, we focus on malicious packages developed in the Python programming language as our primary research subject. By conducting a comprehensive analysis at the source code level, we aim to identify and characterize the distinctive features of data-stealing malicious packages. This approach allows us to gain deeper insights into their behavior, implementation techniques, and potential impact on software security.

Research Questions. We design this study to answer the following five research questions.

- **RQ1 Prevalence Study:** What are the trends in downloads of malicious data-stealing packages in recent years?
- **RQ2 Pattern Study:** What are the attack patterns employed by data-stealing packages?
- **RQ3 Disguise Characteristic Study:** What are the disguise techniques employed by data-stealing packages at the source code level?
- **RQ4 Objectives Study:** What objectives do data-stealing packages aim to achieve?
- **RQ5 Detection Effectiveness study:** What is the current detection rate of data-stealing malicious packages by malicious package detection software and websites?

3.1 Data Stealing Package Collection

We constructed the dataset as follows.For malicious packages,we collected malicious samples from two sources.We selected 2,483 malicious packages from the pypi section of Backstabber's Knife Collection.We also sampled 3,695 packages out of a total of 5,874 packages from pypi_malregistry,ultimately formed a Python malicious package dataset with a capacity of 6,178. We randomly selected 735 packages as our research samples from the database,with a confidence level of 99%. Within the sample, we manually screened data-stealing packages according to the following definitions:

system info	software info	OS
		hostname
		username
		computer name
		cwd
	hardware info	uptime
		RAM
		MAC
		IP
		CPU
		GPU
		UUID
personal info	location	
	email	
	phone number	
	2FA/MFA	
	name	
website/software/browser info	authentication info	encrypted_key
		cookies
		tokens
	user info	
	browsers history	
user behavior data	screenshot	
	videocapture	
	click data	
financial info	Cryptocurrencies wallets	
	credit-card	
	payment methods	

Table 1: Sensitive Information Statistics

A data-stealing malware package is a type of malware designed to covertly access and transfer a user's sensitive information or data without authorization.

Specifically, we use three indicators to determine whether a malicious package is a data-stealing package: obtaining sensitive information and transmitting sensitive information. Our classification of sensitive information is shown in Table 1.If a package obtains the above sensitive information and transmits it, we will mark it as data-stealing.

After screening,we marked 83 data-stealing packages.Concurrently, we systematically recorded and classified the behavioral characteristics of the package to support further research. These characteristics encompassed the transfer function (*i.e.*, the library or platform used for transmission), the transmission mode (*i.e.*, the request method employed), and the info-type (*i.e.*, the type of information stolen), among others.

3.2 Research Questions

In this study, we set five research questions to explore the popularity trends and code features of data-stealing packages.

RQ1 Prevalence Study. In this study, we utilized the BigQuery Sandbox to methodically collect extensive data on the total downloads of classified

data-stealing packages from 2017 to 2024. By examining the download trends of these malicious packages over this seven-year period, we aim to uncover shifts in their popularity. This analysis allows us to gain insights into how the severity and evolving trends of data-stealing threats have changed over time, providing a clearer understanding of their impact and reach within the software ecosystem.

RQ2 Pattern Study. In this study, we perform an in-depth, code-level analysis of the classified data-stealing packages to dissect their underlying mechanisms and operational behaviors. By systematically examining the source code, we identify and categorize the various attack modes employed by these packages. Our goal is to create clear and organized classifications of the attack patterns used by this type of malicious software. Through careful analysis, we aim to explain how these threats work technically and to provide a simple, structured way to understand their behavior and tactics.

RQ3 Disguise Characteristic Study. In this study, we conducted a comprehensive and detailed investigation into the disguise characteristics of data-stealing malicious packages. Our research involved a meticulous examination of the metadata associated with each package, including attributes such as descriptions ,author name and email. By carefully analyzing this metadata, we assessed its validity and determined whether it was intentionally manipulated to mimic legitimate, benign packages. Furthermore, we recorded both the specific goals behind these disguises and the techniques used to accomplish them. These methods encompass a range of sophisticated tactics, such as encrypting stolen data to evade detection, integrating anti-detection mechanisms to bypass security systems, and designing custom download processes to conceal malicious activities.

RQ4 Objectives Study. In this study, we carried out an exploration of the objectives that data-stealing malicious packages aim to achieve. Our research focused on identifying the primary goals driving the development and deployment of these packages, ranging from financial gain to personal information gain. By analyzing the behavior and functionalitynof these packages, we sought to uncover the underlying motivations behind their creation and distribution. Through careful examination of the information stolen and additional means of attack, we categorized the objectives into distinct types, such as stealing sensitive user data, obtaining financial benefits, or compromising system integrity for further exploitation. By shedding light on these objectives, our study aims to contribute to the development of more targeted and effective countermeasures, ultimately enhancing the ability to detect, prevent, and mitigate the impact of such malicious activities.

RQ5 Detection Effectiveness study.

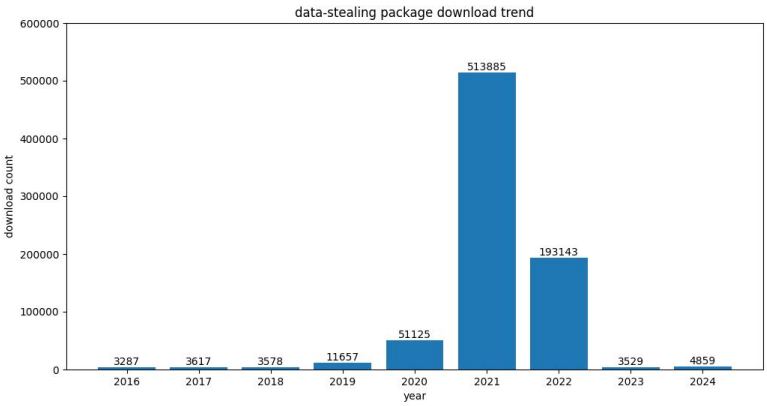

Fig. 1: Download Trends of Data-Stealing Packages (2017–2024)

4 Results

4.1 Prevalence Study (RQ1)

By collecting and counting the download data of the classified data-stealing packages, we generated a trend chart covering the period from 2017 to 2024, as depicted in Fig.1. The statistical chart shows that the package downloads gradually increased from 2017 to 2022, with a larger growth rate in 2021 and 2022, reaching a peak in 2021,then declined in 2023 and 2024.

We found that 2021 was a period of outbreak of malicious packages.At the same time, the download volume in 2020 cannot be ignored.In 2021, software supply chain attacks became a major trend in cybercrime, and attackers began to exploit open source software repositories (such as PyPI, npm, and RubyGems) on a large scale. Furthermore,since PyPI prioritizes installing the latest versions of libraries by default, attackers can upload malicious updates on PyPI and trick developers into downloading them. The 2020-2021 SolarWinds hack was one of the most serious supply chain attacks in recent years.The success of hackers using supply chain vulnerabilities to break into the U.S. government and companies had brought supply chain attacks into the public eye, and as a result, the hacker community had begun to use PyPI more frequently to spread malware. Additionally, on June 23,2022, the OSCS,open source security community monitoring found that the PyPI official repository was uploaded by the attacker mega707 with more than 150 malicious phishing packages such as ‘agoric-sdk‘ ‘datashare‘ ‘datadog-agent‘.

The decrease in downloads in 2023 and 2024 is speculated to be caused by restrictions imposed by the PyPI. On May 25, 2023, PyPI announced that every account that maintains any project or organization on PyPI will be required to enable Two-factor authentication (2FA) on their account by the end of 2023.And On January 1, 2024, PyPI requires 2FA for all users. Prior to this measure, if someone used a simple or reused password, an attacker could

take control of the maintainer's account and begin acting as that user, while installing and executing software on unsuspecting users' computers. Enabling 2FA makes it almost impossible for an attacker to directly control the PyPI maintainer's account even if they steal the password, significantly reducing the risk of legitimate accounts being hijacked to upload malicious packages. This is critical to the security of the PyPI ecosystem and helps protect developers and users from malware.

4.2 Data Stealing Pattern Study (RQ2)

Pattern	Description	Percentage
1	Decrypt Authentication Information->steal personal, system info->packaging info->transmit->extra attack	3.6%
2	Decrypt Authentication Information->steal system,personal->(encrypt info)->packaging info->transmit	7.2%
3	steal system info->packaging info->(encrypt info)->transmit	72.3%
4	steal system info->transmit->remote download->execute downloaded files	10.8%
5	steal system info->transmit->extra attack	1.2%
6	steal financial info->packaging info->transmit	1.2%
7	steal personal,financial info->packaging info->transmit->extra attack	1.2%
8	steal system,software info(configuration file)->transmit	1.2%
9	steal authentification info->transmit	1.2%

Table 2: Attack Pattern Classification and Statistics

4.3 Disguise Characteristic Study (RQ3)

4.4 Stealing Objective Study (RQ4)

4.5 Malware Detection Effectiveness Evaluation (RQ5)

5 Related Work

Malicious Package Detection.

Security Threats in OSS Supply Chain.

5.1 Threats and Limitations

6 Conclusion and Future Work

xxxx [Cappos et al.(2008)]

Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No. 62332005, 62372114 and 62402342).

References

[Agten et al.(2015)] Pieter Agten, Wouter Joosen, Frank Piessens, and Nick Nikiforakis. 2015. Seven months' worth of mistakes: A longitudinal study of typosquatting abuse. In *Proceedings of the 22nd Network and Distributed System Security Symposium*.

[Alfadel et al.(2023)] Mahmoud Alfadel, Diego Elias Costa, and Emad Shihab. 2023. Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering* 28 (2023).

[Bagmar et al.(2021)] Aadesh Bagmar, Josiah Wedgwood, Dave Levin, and Jim Purtilo. 2021. I know what you imported last summer: A study of security threats in the python ecosystem. *arXiv preprint arXiv:2102.06301* (2021).

[Cao and Dolan-Gavitt(2022)] Alan Cao and Brendan Dolan-Gavitt. 2022. What the Fork? Finding and Analyzing Malware in GitHub Forks. In *Proceedings of the Workshop on Measurements, Attacks, and Defenses for the Web*.

[Cappos et al.(2008)] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. 2008. A look in the mirror: Attacks on package managers. In *Proceedings of the 15th ACM conference on Computer and communications security*. 565–574.

[Duan et al.(2020)] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. 2020. Towards measuring supply chain attacks on package managers for interpreted languages. In *Proceedings of 27th Annual Network and Distributed System Security Symposium*.

[Enck and Williams(2022)] William Enck and Laurie Williams. 2022. Top five challenges in software supply chain security: Observations from 30 industry and government organizations. *IEEE Security & Privacy* 20, 2 (2022), 96–100.

[Fass et al.(2019)] Aurore Fass, Michael Backes, and Ben Stock. 2019. Jstap: A static pre-filter for malicious javascript detection. In *Proceedings of the 35th Annual Computer Security Applications Conference*. 257–269.

[Fass et al.(2018)] Aurore Fass, Robert P Krawczyk, Michael Backes, and Ben Stock. 2018. Jast: Fully syntactic detection of malicious (obfuscated) javascript. In *Proceedings of the 15th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 303–325.

[Ferreira et al.(2021)] Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. Containing malicious package updates in npm with a lightweight permission system. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering*. 1334–1346.

[Garrett et al.(2019)] Kalil Garrett, Gabriel Ferreira, Limin Jia, Joshua Sunshine, and Christian Kästner. 2019. Detecting suspicious package updates. In *Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results*. 13–16.

[Gkortzis et al.(2021)] Antonios Gkortzis, Daniel Feitosa, and Diomidis Spinellis. 2021. Software reuse cuts both ways: An empirical analysis of its relationship with security vulnerabilities. *Journal of Systems and Software* 172 (2021).

[Gonzalez et al.(2021)] Danielle Gonzalez, Thomas Zimmermann, Patrice Godefroid, and Max Schäfer. 2021. Anomalicious: Automated detection of anomalous and potentially malicious commits on github. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice*. 258–267.

[Goyal et al.(2018)] Raman Goyal, Gabriel Ferreira, Christian Kästner, and James Herbsleb. 2018. Identifying unusual commits on GitHub. *Journal of Software: Evolution and Process* 30, 1 (2018).

[Gu et al.(2023)] Yacong Gu, Lingyun Ying, Huajun Chai, Chu Qiao, Haixin Duan, and Xing Gao. 2023. Continuous Intrusion: Characterizing the Security of Continuous Integration Services. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1561–1577.

[Gu et al.(2022)] Yacong Gu, Lingyun Ying, Yingyuan Pu, Xiao Hu, Huajun Chai, Ruimin Wang, Xing Gao, and Haixin Duan. 2022. Investigating Package Related Security Threats in Software Registries. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1151–1168.

[Hu et al.(2020)] Yangyu Hu, Haoyu Wang, Ren He, Li Li, Gareth Tyson, Ignacio Castro, Yao Guo, Lei Wu, and Guoai Xu. 2020. Mobile app squatting. In *Proceedings of The Web Conference 2020*. 1727–1738.

[Huang et al.(2022)] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.

[Huang et al.(2024)] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. 2024. SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[Kaplan and Qian(2021)] Berkay Kaplan and Jingyu Qian. 2021. A survey on common threats in npm and pypi registries. In *Proceedings of the Second International Workshop on Deployable Machine Learning for Security Defense*. 132–156.

[Kenton and Toutanova(2019)] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 4171–4186.

[Khan et al.(2015)] Mohammad Taha Khan, Xiang Huo, Zhou Li, and Chris Kanich. 2015. Every second counts: Quantifying the negative externalities of cybercrime via typosquatting. In *Proceedings of the 2015 IEEE Symposium on Security and Privacy*. 135–150.

[Kintis et al.(2017)] Panagiotis Kintis, Najmeh Miramirkhani, Charles Lever, Yizheng Chen, Rosa Romero-Gómez, Nikolaos Pitropakis, Nick Nikiforakis, and Manos Antonakakis. 2017. Hiding in plain sight: A longitudinal study of combosquatting abuse. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 569–586.

[Koishybayev et al.(2022)] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the Security of Github {CI} Workflows. In *Proceedings of the 31st USENIX Security Symposium*. 2747–2763.

[Ladisa et al.(2023a)] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023a. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *Proceedings of the 2023 IEEE Symposium on Security and Privacy*. 1509–1526.

[Ladisa et al.(2023b)] Piergiorgio Ladisa, Serena Elisa Ponta, Nicola Ronzoni, Matias Martinez, and Olivier Barais. 2023b. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Proceedings of the 39th Annual Computer Security Applications Conference*. 71–82.

[Lakshmanan(2022)] Ravie Lakshmanan. 2022. Malware Strains Targeting Python and JavaScript Developers Through Official Repositories. Retrieved May 30, 2023 from `https://thehackernews.com/2022/12/malware-strains-targeting-python-and.html`

[Lamb and Zacchiroli(2021)] Chris Lamb and Stefano Zacchiroli. 2021. Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software* 39, 2 (2021), 62–70.

[Liang et al.(2021)] Genpei Liang, Xiangyu Zhou, Qingyu Wang, Yutong Du, and Cheng Huang. 2021. Malicious Packages Lurking in User-Friendly Python Package Index. In *Proceedings of the IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications*. 606–613.

[Liu et al.(2022a)] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022a. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*. 672–684.

[Liu et al.(2022b)] Guannan Liu, Xing Gao, Haining Wang, and Kun Sun. 2022b. Exploring the Unchartered Space of Container Registry Typosquatting. In *Proceedings of the 31st USENIX Security Symposium*. 35–51.

[Liu et al.(2019)] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[Microsoft(2020)] Microsoft. 2020. OSS Detect Backdoor. Retrieved May 30, 2023 from `https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor`

[Muncaster(2023)] Phil Muncaster. 2023. Hundreds Malicious Packages NPM. Retrieved May 30, 2023 from `https://www.infosecurity-magazine.com/news/hundreds-malicious-packages-npm/`

[Ohm et al.(2022)] Marc Ohm, Felix Boes, Christian Bungartz, and Michael Meier. 2022. On the feasibility of supervised machine learning for the detection of malicious software packages. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*. 1–10.

[Ohm et al.(2020a)] Marc Ohm, Lukas Kempf, Felix Boes, and Michael Meier. 2020a. Supporting the detection of software supply chain attacks through unsupervised signature generation. *arXiv preprint arXiv:2011.02235* (2020).

[Ohm et al.(2020b)] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020b. Backstabber's knife collection: A review of open source software supply chain attacks. In *Proceedings of the 17th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. 23–43.

[Ponta et al.(2018)] Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution*. 449–460.

[Prana et al.(2021)] Gede Artha Azriadi Prana, Abhishek Sharma, Lwin Khin Shar, Darius Foo, Andrew E Santosa, Asankhaya Sharma, and David Lo. 2021. Out of sight, out of mind? How vulnerable dependencies affect open-source projects. *Empirical Software Engineering* 26 (2021).

[Raffel et al.(2020)] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.

[Ruohonen et al.(2021)] Jukka Ruohonen, Kalle Hjerppe, and Kalle Rindell. 2021. A Large-Scale Security-Oriented Static Analysis of Python Packages in PyPI. In *Proceedings of the 18th International Conference on Privacy, Security and Trust*. 1–10.

[Sejfia and Schäfer(2022)] Adriana Sejfia and Max Schäfer. 2022. Practical automated detection of malicious npm packages. In *Proceedings of the 44th International Conference on Software Engineering*. 1681–1692.

[Sun et al.(2024)] Xiaobing Sun, Xingan Gao, Sicong Cao, Lili Bo, Xiaoxue Wu, and Kaifeng Huang. 2024. 1+1>2: Integrating Deep Code Behaviors with Metadata Features for Malicious PyPI Package Detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[Szurdi et al.(2014)] Janos Szurdi, Balazs Kocso, Gabor Cseh, Jonathan Spring, Mark Felegyhazi, and Chris Kanich. 2014. The long "taile" of typosquatting domain names. In *Proceedings of the 23rd USENIX Security Symposium*. 191–206.

[Tahir et al.(2018)] Rashid Tahir, Ali Raza, Faizan Ahmad, Jehangir Kazi, Fareed Zaffar, Chris Kanich, and Matthew Caesar. 2018. It's all in the name: Why some URLs are

more vulnerable to typosquatting. In *Proceedings of the 2018 IEEE Conference on Computer Communications*. 2618–2626.

[Taylor et al.(2020)] Matthew Taylor, Ruturaj Vaidya, Drew Davidson, Lorenzo De Carli, and Vaibhav Rastogi. 2020. Defending against package typosquatting. In *Proceedings of the 14th International Conference on Network and System Security*. 112–131.

[virustotal(2023)] virustotal. 2023. *Virustotal*. Retrieved May 1, 2023 from `https://www.virustotal.com/gui/home/upload`

[Vu(2020)] Duc-Ly Vu. 2020. Bandit4Mal. Retrieved May 30, 2023 from `https://github.com/lyvd/bandit4mal`

[Vu et al.(2021)] Duc-Ly Vu, Fabio Massacci, Ivan Pashchenko, Henrik Plate, and Antonino Sabetta. 2021. Lastpymile: identifying the discrepancy between sources and packages. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 780–792.

[Vu et al.(2023)] Duc-Ly Vu, Zachary Newman, and John Speed Meyers. 2023. Bad Snakes: Understanding and Improving Python Package Index Malware Scanning. In *Proceedings of the 45th International Conference on Software Engineering*.

[Vu et al.(2020a)] Duc Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020a. Towards using source code repositories to identify software supply chain attacks. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 2093–2095.

[Vu et al.(2020b)] Duc-Ly Vu, Ivan Pashchenko, Fabio Massacci, Henrik Plate, and Antonino Sabetta. 2020b. Typosquatting and combosquatting attacks on the python ecosystem. In *Proceedings of the 2020 2020 IEEE European Symposium on Security and Privacy Workshops*. 509–514.

[Warehouse(2020)] Warehouse. 2020. Malware Checks. Retrieved May 1, 2023 from `https://warehouse.pypa.io/development/malware-checks.html`

[Wenbo et al.(2023)] Guo Wenbo, Xu Zhengzi, Liu Chengwei, Huang Cheng, Fang Yong, and Liu Yang. 2023. An Empirical Study of Malicious Code In PyPI Ecosystem. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.

[Wentao et al.(2023)] Liang Wentao, Ling Xiang, Wu Jingzheng, Luo Tianyue, and Yanjun Wu. 2023. A Needle is an Outlier in a Haystack: Hunting Malicious PyPI Packages with Code Clustering. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.

[Wheeler(2005)] David A Wheeler. 2005. Countering trusting trust through diverse double-compiling. In *Proceedings of the 21st Annual Computer Security Applications Conference*. 33–48.

[Xu et al.(2022)] Meiqiu Xu, Ying Wang, Shing-Chi Cheung, Hai Yu, and Zhiliang Zhu. 2022. Insight: Exploring Cross-Ecosystem Vulnerability Impacts. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.

[Ye et al.(2017)] Yanfang Ye, Tao Li, Donald Adjeroh, and S Sitharama Iyengar. 2017. A survey on malware detection using data mining techniques. *ACM Computing Surveys (CSUR)* 50, 3 (2017), 1–40.

[Zahan et al.(2022)] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. 2022. What are weak links in the npm supply chain?. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. 331–340.

[Zerouali et al.(2022)] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. 2022. On the impact of security vulnerabilities in the npm and rubygems dependency networks. *Empirical Software Engineering* 27 (2022).

[Zimmermann et al.(2019)] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. 2019. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proceedings of the 28th USENIX Security Symposium*. 995–1010.